

Week 1(b): Demonstrate the Data-Preprocessing techniques by taking real dataset.

#Step 1: Importing Libraries and the Dataset

importing libraries

```
import pandas as pd
import scipy
import numpy as np
from sklearn.preprocessing import MinMaxScaler
import seaborn as sns
import matplotlib.pyplot as plt
```

#Step 2: Load the dataset

#Dataset link: [<https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>]

Load the dataset

```
df = pd.read_csv('diabetes.csv')
print(df.head())
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

#Check the data info

```
df.info( )
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 768 entries, 0 to 767
```

```
Data columns (total 9 columns):
```

#	Column	Non-Null Count	Dtype
0	Pregnancies	768 non-null	int64
1	Glucose	768 non-null	int64
2	BloodPressure	768 non-null	int64
3	SkinThickness	768 non-null	int64

4	Insulin	768	non-null	int64
5	BMI	768	non-null	float64
6	DiabetesPedigreeFunction	768	non-null	float64
7	Age	768	non-null	int64
8	Outcome	768	non-null	int64

dtypes: float64(2), int64(7)

memory usage: 54.1 KB

#Check the null values using df.isnull()

df.isnull().sum()

Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

#Step 3: Statistical Analysis

#In statistical analysis, first, we use the df.describe() which will give a descriptive overview of the dataset.

df.describe()

	Pregnancies	Glucose	BloodPressure	SkinThickness
Insulin \				
count	768.000000	768.000000	768.000000	768.000000
768.000000				
mean	3.845052	120.894531	69.105469	20.536458
79.799479				
std	3.369578	31.972618	19.355807	15.952218
115.244002				
min	0.000000	0.000000	0.000000	0.000000
0.000000				
25%	1.000000	99.000000	62.000000	0.000000
0.000000				
50%	3.000000	117.000000	72.000000	23.000000
30.500000				
75%	6.000000	140.250000	80.000000	32.000000
127.250000				
max	17.000000	199.000000	122.000000	99.000000
846.000000				

	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000
mean	31.992578	0.471876	33.240885	0.348958
std	7.884160	0.331329	11.760232	0.476951

min	0.000000	0.078000	21.000000	0.000000
25%	27.300000	0.243750	24.000000	0.000000
50%	32.000000	0.372500	29.000000	0.000000
75%	36.600000	0.626250	41.000000	1.000000
max	67.100000	2.420000	81.000000	1.000000

Note: The above table shows the count, mean, standard deviation, min, 25%, 50%, 75%, and max values for each column. When we carefully observe the table we will find that. Insulin, Pregnancies, BMI, Blood Pressure columns has outliers.

Step 4: Normalization or Standardization

1. Normalization
2. MinMaxScaler scales the data so that each feature is in the range [0, 1].
3. Rescale your data using scikit-learn using the MinMaxScaler.

```
# Initialising the MinMaxScaler
scaler = MinMaxScaler(feature_range=(0, 1))
# learning the statistical parameters for each of the data and transforming
rescaledX = scaler.fit_transform(X)
rescaledX[:5]

array([[0.35294118, 0.74371859, 0.59016393, 0.35353535, 0.50074516, 0.23441503, 0.48333333, 1.005882353, 0.42713568, 0.54098361, 0.29292929, 0.39642325, 0.11656704, 0.16666667, 0.47058824, 0.91959799, 0.52459016, 0.34724292, 0.25362938, 0.18333333, 0.05882353, 0.44723618, 0.54098361, 0.23232323, 0.11111111, 0.41877794, 0.03800171, 0.0.68844221, 0.32786885, 0.35353535, 0.19858156, 0.64232489, 0.94363792, 0.2, 1.]])
```

Standardization . Standardization is a useful technique to transform attributes with a Gaussian distribution and differing means and standard deviations to a standard Gaussian distribution with a mean of 0 and a standard deviation of 1. . We can standardize data using scikit-learn with the StandardScaler class. . It works well when the features have a normal distribution or when the algorithm being used is not sensitive to the scale of the features

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler().fit(df)
rescaledX = scaler.transform(df)
rescaledX[:5]

array([[ 0.63994726,  0.84832379,  0.14964075,  0.90726993, -0.69289057,
         0.20401277,  0.46849198,  1.4259954 ,  1.36589591],
       [-0.84488505, -1.12339636, -0.16054575,  0.53090156, -
```

```
0.69289057,  
    -0.68442195, -0.36506078, -0.19067191, -0.73212021],  
    [ 1.23388019,  1.94372388, -0.26394125, -1.28821221, -  
0.69289057,  
    -1.10325546,  0.60439732, -0.10558415,  1.36589591],  
    [-0.84488505, -0.99820778, -0.16054575,  0.15453319,  
0.12330164,  
    -0.49404308, -0.92076261, -1.04154944, -0.73212021],  
    [-1.14185152,  0.5040552 , -1.50468724,  0.90726993,  
0.76583594,  
    1.4097456 ,  5.4849091 , -0.0204964 ,  1.36589591]]))
```